Agent Copilot: build voice agents like software, watch them at scale

Prosper challenge, Phase 2 · Feliu Serra Burriel · October 2026

1 The problem

Prosper's deployment team spends its time on two manual workflows: turning a clinic's guidelines into a working voice agent, and refining agents from issues that clients flag or that show up in calls. We asked Prosper three questions, and the answers set the scope:

- Most time goes to turning rules into an agent and checking that it behaves, not to getting the rules from the client.
- Agents start from a template per use case and are always customized. Between two small clinics, what changes is mostly tables (appointment types, which provider does which, which insurance covers which visit, the fields needed to register a patient) plus flow and tone.
- One person manages fewer than 10 agents, and an agent can take 50,000 calls a month. How to find problems in them is left to us.

So there are two bottlenecks. **The team's:** building an agent and checking it. **Each agent's:** the places in its graph where calls stall, leak or go wrong. At up to 500,000 calls a month per person nobody can listen, and the scarce resource is attention, not model cost.

Thesis. Coding agents work because code comes with a spec, tests they can run and bug reports with evidence. Voice agents have none of these, which is why both workflows are manual. The Copilot creates them, then works the loop a coding agent works (Figure 1).

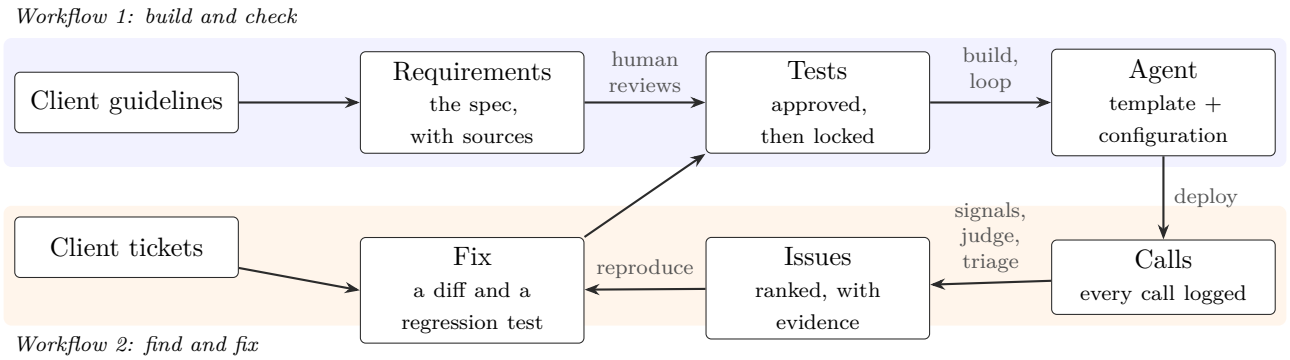


Figure 1: The loop. A ticket without a call becomes a new or changed requirement.

2 Workflow 1: from guidelines to a checked agent

- **Clinic rules are data, not prompts.** Tables and numbers (appointment types, providers per type, coverage, required fields, the cancellation fee) go into a validated clinic configuration that the tools enforce: `book_appointment` refuses a visit the provider does not do, with a reason the model can act on. A new clinic is a configuration plus a small diff on the scheduling template, not a new graph, and the human reviews the configuration as tables.
- **Requirements are the spec.** The Copilot turns guidelines into numbered requirements, each citing the sentence it came from, plus questions for the client where the guidelines are silent, each with the safe default used meanwhile. A healthcare baseline is always added: emergencies, no medical advice, identity before any booking, a human on request.
- **The human reviews tests, not prompts.** Checking 20 one-line tests against the guidelines takes minutes; reading every prompt in a graph takes an hour. Tests are approved before the graph exists and then locked, so the Copilot cannot pass by editing them.
- **Exact checks first, a judge for the rest.** A small model plays a caller with a persona, a goal and facts, through a text runner that follows the voice runtime's rules. What the agent did is checked exactly: tools called, their arguments and order, and how the call ended. A model judge checks what it said: read-backs, privacy, emergencies, medical advice, and that every time and fact came from a tool.

- **Loop, then one diff.** The Copilot edits through validated operations, reruns the failing tests (at most 3 rounds) and proposes one diff to accept or reject like a pull request; while tests fail, the proposal is blocked and says why (the broken link per failure, client questions, tests it doubts). The loop has not paid off yet: $12.38 against $2.11 without it, for the same score.

Why "all tests pass" means something. A model writes the agent, plays the caller and grades the call, so it can agree with itself. Against that: a 20-test reference suite the Copilot never sees; mutation testing (12 of 15 broken agents fail a test; of the 3 that pass, two drop a rule that holds anyway, and one skips the spelling, which a simulated caller can check by reading but a real one cannot); a judge calibrated on hand-labelled calls (below 90% agreement, a rule becomes an exact check); a check that no test data (names, birth dates) appears in the agent; and, designed but not yet run, a second clinic written by a person and kept out of the repo until a single final run.

3 Workflow 2: finding problems in 500,000 calls a month

A call through our agents is not a free-form conversation: it is a path through a known graph. Every call already emits its node transitions, so every call is a row in an event log, with no model call at all. Detection starts there, and the graph tells us what kind of failure we are looking at:

Shape	What it looks like	Found by
Stall	back to the same or an earlier node, or a long visit: a name speech-to-text keeps garbling	transitions
Leak	hang-ups in a node where callers should not hang up	transitions
Detour	an edge taken more often than it should be, such as needless transfers	transitions
Tool error	a tool keeps refusing: a booking field the agent never asks for	tool results
Silent failure	the path looks right, the outcome is wrong: the wrong name, an invented slot	judge, clinic records

Three tiers, cheapest first:

1. **Signals on every call, no model.** Per node: entries, exits per edge, turns spent, re-entries, hang-ups, transfers, tool errors. From the clinic system: the same number calling back within 48 hours, a booking cancelled or changed within a day, a new patient record that matches an existing one's birth date and phone under another name. That last one is the ticket "Mrs. Lopez was booked under someone else's name", found without reading a transcript.
2. **A model judge on a sample (built).** Calls flagged by tier 1, plus a random sample, are checked against the clinic's requirements and a general checklist. Each issue gets a severity, the requirement, the node and quotes. A random sample of 300 calls contains at least one example of any failure that affects 1% of calls with 95% probability ($1 - 0.99^{300}$); for an agent at 50,000 calls a month that is under 3% of a week's traffic.
3. **Triage.** Issues are grouped by node, requirement and shape, and ranked by severity, calls affected and trend. The output is a short list per person, not a dashboard, for example (from the synthetic logs): *Riverside, Confirm name: re-entries at 8.9% of calls since version 3, against 2.0% in version 2 and 2.4% at other clinics, about 680 more calls a week, 3 example calls.*



Figure 2: The call heatmap on the synthetic logs (crop): Riverside, version 3 against 2, re-entries. Confirm name, the planted stall, is the only tinted node (8.9% against 2.0%). Edge width is traffic; right, the ranked issues.

Baselines make a rate mean something. Hang-ups after a cancellation are normal. Each node is compared with itself in the previous version, so an accepted change that makes things worse raises an alarm, and with the same template node in other clinics: because agents come from a template, Confirm name at clinic A can be compared with Confirm name everywhere else. Templates give a fleet-wide baseline for free. A two-proportion test with a minimum effect size keeps a busy agent from flagging tiny differences and a quiet one from flagging noise.

The heatmap is the view, not the detector. Opening a ranked issue shows the canvas with nodes tinted by their distance from the baseline, edges drawn by traffic, and collapsed stages rolling up their nodes (Figure 2). Designed next: *fix from this issue* reproduces the call in the text runner from the words speech-to-text heard, finds the turn that decided it, fixes it, and keeps the reproduction as a regression test.

4 Scope: built, mocked, designed, left out

Part	Status	Why
Graph editor, test calls with live node highlighting	Built	Phase 1: everything else sits on it.
Clinic tools and calendar	Mocked	No access to a real clinic system; tools are where clinic rules are enforced.
Text runner and test format	Built	A simulated call costs cents and runs in seconds.
Clinic configuration and template	Built	The interviewer's answer 2: clinics differ mostly in tables.
Spec, tests, build, loop	Built	Workflow 1, where the team's time goes. Run from the Copilot panel or `make`.
Production calls	Mocked	No real call data: 85,000 synthetic call logs from 3 clinics on one template (one at 50,000 calls a month), defects planted at known rates.
Signals, triage, heatmap	Built	Workflow 2, the part the brief left open.
Judge on a sample	Built	Failures the path cannot show: a wrong name, an invented slot.
Fix from an issue: reproduce, fix, regression test	Designed	Closes the loop. No forecast of how the call would have gone.
Intake call and client checklist	Left out	Getting the rules is not the bottleneck (answer 1).
Telephony, rollout and rollback, auth, PHI redaction, other languages	Left out	Production infrastructure, not the Copilot.

5 Results and limits

Measure	Result
Reference suite, Copilot-built agent for clinic A. Misses: two read-backs (since fixed in the template), two insurance questions, a request outside hours	13 of 20 (target 18)
Broken agents caught by the tests; judge agreement with hand labels	12 of 15; 32 of 32
Planted defects found by signals and triage; false alarms	6 of 6; none in 18 clinic-weeks
Silent failures in 22 mock calls found by the judge; false alarms	8 of 10; 2
Spec step on clinic A's guidelines: requirements, tests, client questions, time	45, 25, 5 in 136 s
Cost: one simulated test call (judge included), the spec step, one build	\$0.09, \$0.38, \$2.11

Where detection stops. One node's re-entries raised over a 2% base, at five sizes and three volumes: the share of 10 seeds flagged within two weeks of the version that caused it (10 false alarms in those 600 clinic-weeks).

Calls a week	+0.5 pt	+1 pt	+2 pt	+3 pt	+5 pt
150	0%	0%	20%	50%	70%
1,500	10%	40%	100%	100%	100%
11,500	60%	100%	100%	100%	100%

What this does not show. Text tests do not see latency, interruptions, audio, or speech-to-text errors other than the ones replayed from real calls. Synthetic logs prove the statistics and the plumbing, not how real callers behave; real rates and baselines are learned in production. In production the final judge is real outcomes: completed bookings, callbacks and hang-ups.

6 Future work: say which tools are missing

The Copilot does not write tools. Each catalog tool stands for an integration with the clinic's scheduling or records system, and the exact tests only mean the same thing on every agent because all agents share the catalog. The Copilot picks tools from it and fits their parameters to the clinic's configuration. What it should add is a report of the tools it needed and did not have, so the gap reaches a person instead of being papered over in a prompt.

- **At build time, next to the client questions.** The build already lists every requirement it leaves uncarried, each with a free-text reason, but the list mixes "the template or the tools already do this" with "the agent cannot do this". The second becomes a kind of its own, *needs a tool*: it names the missing capability ("text the patient a confirmation", "put the caller on a waitlist"), cites the guideline sentence and states what the agent does in the meantime: says it cannot, takes a message, or transfers. The reviewer sees it next to the questions for the client.
- **In production, on the heatmap.** The judge on a sample (tier 2) tags calls where the caller asked for something the agent had no tool for, along with the transfer or hang-up that followed. Triage counts these by node and by request across the fleet, which is meaningful because every clinic runs the same template: for example, a waitlist asked for at Find slots at every clinic, with its share of calls and example calls. The heatmap gains one more layer: where callers ask for what the agent cannot do.
- **The output is a ranked integration backlog.** Each entry lists the requirements that need it, the clinics that asked, the calls it would have changed, and quotes. Adding a tool stays a person's decision and an engineer's job. The Copilot shows which one is worth building next.